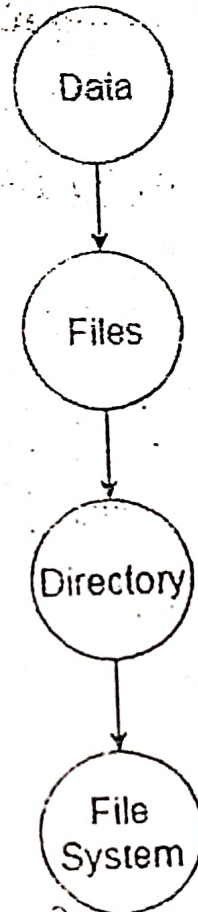


File System Implementation in Operating System

File

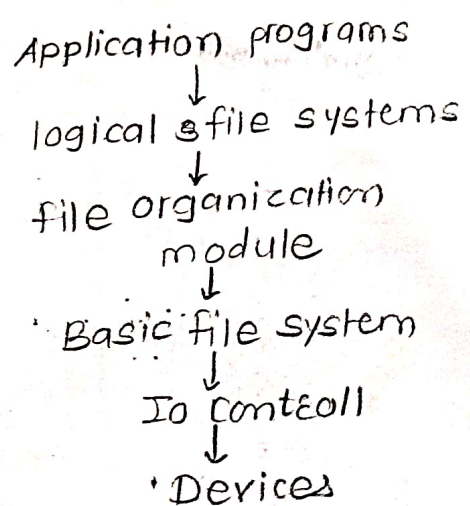
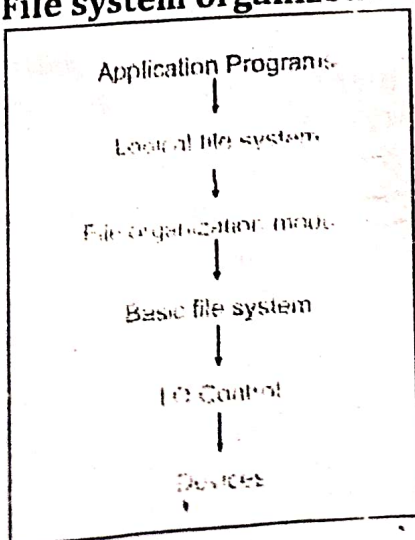
A file can be defined as a data structure which stores the sequence of records. Files are stored in a file system, which may exist on a disk or in the main memory. Files can be simple (plain text) or complex (specially-formatted).

The collection of files is known as Directory. The collection of directories at the different levels, is known as File System.



A file is a collection of related information. The file system resides on secondary storage and provides efficient and convenient access to the disk by allowing data to be stored, located, and retrieved.

File system organized in many layers :



- **I/O Control level -**

Device drivers acts as interface between devices and Os they help to transfer data between disk and main memory. It takes block number as input and as output it gives level hardware specific instruction.

/li>

- **Basic file system -**

It Issues general commands to device driver to read and write physical blocks on disk. It manages the memory buffers and caches. A block in buffer can hold the contents of the disk block and cache stores frequently used file system metadata.

- **(File organization Module -)**

It has information about files, location of files and their logical and physical blocks. Physical blocks do not match with logical numbers of logical block numbered from 0 to N. It also has a free space which tracks unallocated blocks.

- **Logical file system -**

It manages metadata information about a file i.e. includes all details about a file except the actual contents of file. It also maintains via file control blocks. File control block (FCB) has information about a file - owner, size, permissions, location of file contents.

Advantages :

1. Duplication of code is minimized.
2. Each file system can have its own logical file system.

Disadvantages :

If we access many files at same time then it results in low performance.

We can implement file system by using two types data structures :

1. On-disk Structures -

Generally they contain information about total number of disk blocks, free disk blocks, location of them and etc. Below given are different on-disk structures :

1. Boot Control Block -

It is usually the first block of volume and it contains information needed to boot an operating system. In UNIX it is called boot block and in NTFS it is called as partition boot sector.

2. Volume Control Block -

It has information about a particular partition ex:- free block count, block size and block pointers etc. In UNIX it is called super block and in NTFS it is stored in master file table.

3. Directory Structure -

They store file names and associated inode numbers. In UNIX, includes file names and associated file names and in NTFS, it is stored in master file table.

4. Per-File FCB -

It contains details about files and it has a unique identifier number to allow association with directory entry. In NTFS it is stored in master file table.

File Control Block (FCB)

File Permissions

File dates (create, access, write)

File owner group, ACL

File Size

File data block pointers to file

File ACLs

2. In-Memory Structure :

They are maintained in main-memory and these are helpful for file system management for caching. Several in-memory structures given below :

5. Mount Table -

It contains information about each mounted volume.

6. Directory-Structure cache -

This cache holds the directory information of recently accessed directories.

7. System wide open-file table -

It contains the copy of FCB of each open file.

8. Per-process open-file table -

It contains information opened by that particular process and it maps with appropriate system wide open-file.

Directory Implementation :

9. Linear List -

It maintains a linear list of filenames with pointers to the data blocks. It is time-consuming also. To create a new file, we must first search the directory to be sure that no existing file has the same name then we add a file at end of the directory. To delete a file, we search the directory for the named file and release the space. To reuse the directory entry either we can mark the entry as unused or we can attach it to a list of free directories.

10. Hash Table -

The hash table takes a value computed from the file name and returns a pointer to the file. It decreases the directory search time. The insertion and deletion process of files is easy. The major difficulty is hash tables are its generally fixed size and hash tables are dependent on hash function on that size.

File System Interface in Operating System

For the majority of users, the file system is the most obvious aspect of any operating system. This provides users the method for storage and access to data as well as programs of the operating system where all the users of the computer system can use it.

The file system consists of 2 distinct parts:

- a collection of files, that store related data, and
- a directory structure, which organizes and provides information about all the files in the system.

In this chapter, you will learn about the different file tribute, concepts of file and its storage along with operations on files.

Table of Contents

File Attributes

A file is named, for the ease of its users and is referred by its name. A name is usually a string of characters like filename.cpp, along with an extension which designates the file format. Some systems (like Linux) distinguish between uppercase and lowercase characters in names, whereas other systems don't. When a file is given a name, it becomes independent of the process, the user and also the system which created it. Let's suppose, one user might make the file filename.cpp, and another user might be editing that file.

File Structure

A File Structure should be according to a required format that the operating system can understand.

- A file has a certain defined structure according to its type.
- A text file is a sequence of characters organized into lines.
- A source file is a sequence of procedures and functions.
- An object file is a sequence of bytes organized into blocks that are understandable by the machine.
- When operating system defines different file structures, it also contains the code to support these file structure. Unix, MS-DOS support minimum number of file structure.

File Type

File type refers to the ability of the operating system to distinguish different types of file such as text files source files and binary files etc. Many operating systems support many types of files. Operating system like MS-DOS and UNIX have the following types of files -

Ordinary files

- These are the files that contain user information.
- These may have text, databases or executable program.
- The user can apply various operations on such files like add, modify, delete or even remove the entire file.

Directory files

- These files contain list of file names and other information related to these files.

Special files

- These files are also known as device files.
- These files represent physical device like disks, terminals, printers, networks, tape drive etc.

These files are of two types -

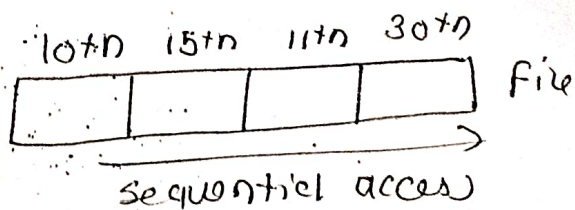
- **Character special files** - data is handled character by character as in case of terminals or printers.

Imp **Block special files** - data is handled in blocks as in the case of disks and tapes.

File Access Mechanisms

File access mechanism refers to the manner in which the records of a file may be accessed. There are several ways to access files -

- Sequential access
- Direct/Random access
- Indexed sequential access

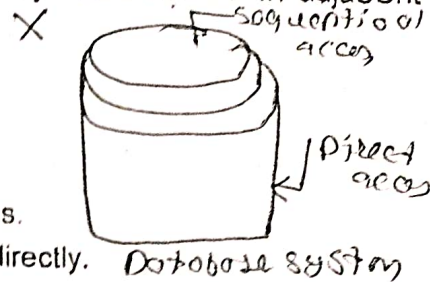


Sequential access

A sequential access is that in which the records are accessed in some sequence, i.e., the information in the file is processed in order, one record after the other. This access method is the most primitive one. Example: Compilers usually access files in this fashion.

Direct/Random access

- Random access file organization provides, accessing the records directly.
- Each record has its own address on the file with by the help of which it can be directly accessed for reading or writing.
- The records need not be in any sequence within the file and they need not be in adjacent locations on the storage medium.



Indexed sequential access

- This mechanism is built up on base of sequential access.
- An index is created for each file which contains pointers to various blocks.
- Index is searched sequentially and its pointer is used to access the file directly.

e by deducing its name. The file's owner may write the file to a compact disk (CD) or send it via an e-mail or copy it across a network, and it could still be called filename.cpp on the destination system.

Fundamental Components of A File

A file's attributes vary from one operating system to another but typically consist of these:

- **Name:** Name is the symbolic file name and is the only information kept in human readable form.
- **Identifier:** This unique tag is a number that identifies the file within the file system; it is in non-human-readable form of the file.
- **Type:** This information is needed for systems which support different types of files or its format.
- **Location:** This information is a pointer to a device which points to the location of the file on the device where it is stored.
- **Size:** The current size of the file (which is in bytes, words, etc.) which possibly the maximum allowed size gets included in this attribute.
- **Protection:** Access-control information establishes who can do the reading, writing, executing, etc.
- **Date, Time & user identification:** This information might be kept for the creation of the file, its last modification and last used. These data might be useful for in the field of protection, security, and monitoring its usage.

File Operations

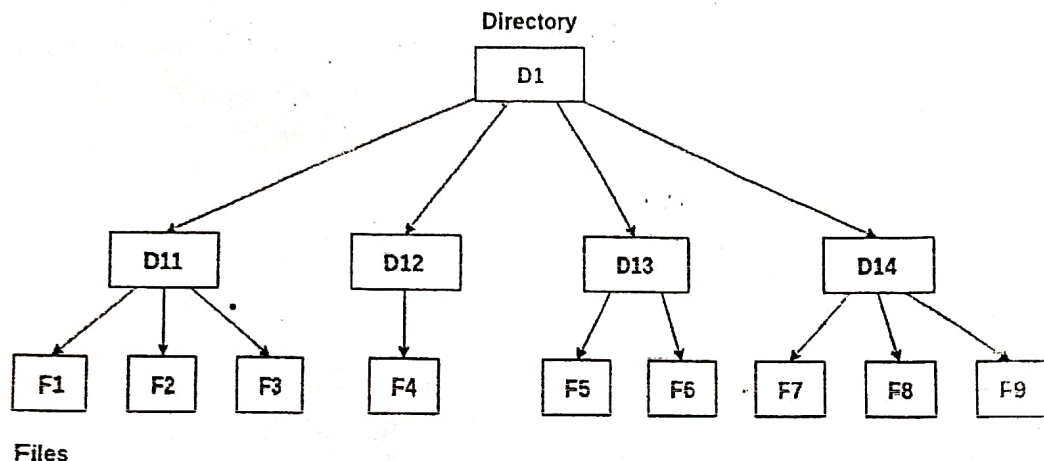
A file is an abstract data type. For defining a file properly, we need to consider the operations that can be performed on files. The operating system can provide system calls to create, write, read, reposition, delete, and truncate files. There are six basic file operations within an Operating system. These are:

- **Creating a file:** There are two steps necessary for creating a file. First, space in the file system must be found for the file. We discuss how to allocate space for the file. Second, an entry for the new file must be made in the directory.
- **Writing a file:** To write to a file, you make a system call specify about both the name of the file along with the information to be written to the file.
- **Reading a file:** To read from a file, you use a system call which specifies the name of the file and where within memory the next block of the file should be placed.

- **Repositioning inside a file:** The directory is then searched for the suitable entry, and the 'current-file-position' pointer is relocating to a given value. Relocating within a file need not require any actual I/O. This file operation is also termed as 'file seek.'
- **Deleting a file:** For deleting a file, you have to search the directory for the specific file. Deleting that file or directory release all file space so that other files can re-use that space.
- **Truncating a file:** The user may wish for erasing the contents of a file but keep the attributes same. Rather than deleting the file and then recreate it, this utility allows all attributes to remain unchanged — except the file length — and let the user add or edit the file content.

Structures of Directory in Operating System

A **directory** is a container that is used to contain folders and files. It organizes files and folders in a hierarchical manner.



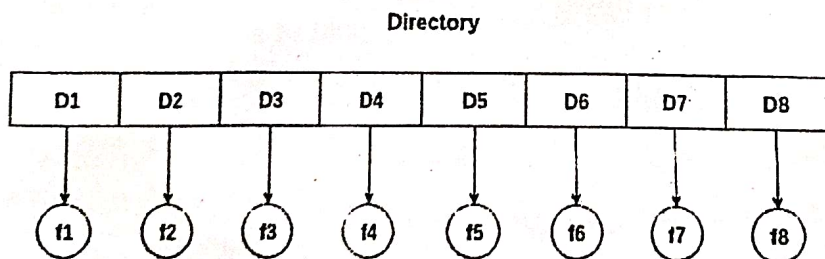
Files

There are several logical structures of a directory, these are given below.

• **Single-level directory -**

The single-level directory is the simplest directory structure. In it, all files are contained in the same directory which makes it easy to support and understand.

A single level directory has a significant limitation, however, when the number of files increases or when the system has more than one user. Since all the files are in the same directory, they must have a unique name. If two users call their dataset test, then the unique name rule is violated.



Files

Advantages:

- Since it is a single directory, so its implementation is very easy.
- If the files are smaller in size, searching will become faster.
- The operations like file creation, searching, deletion, updating are very easy in such a directory structure.

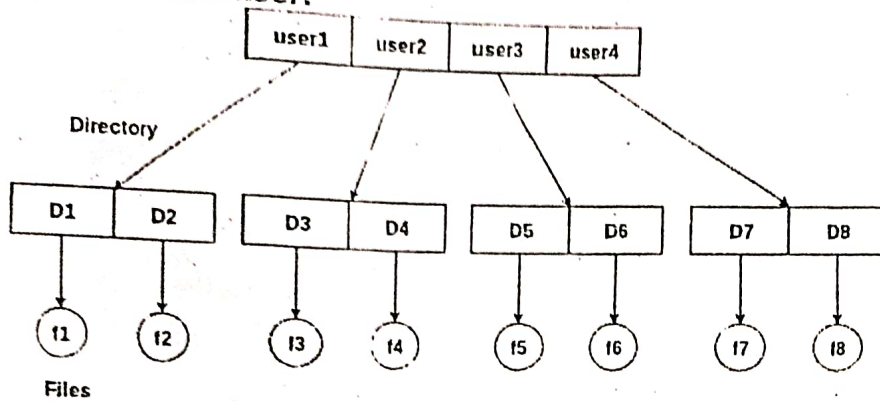
Disadvantages:

- There may be a chance of name collision because two files can not have the same name.
- Searching will become time taking if the directory is large.
- This can not group the same type of files together.

• Two-level directory -

As we have seen, a single level directory often leads to confusion of files names among different users. the solution to this problem is to create a separate directory for each user.

In the two-level directory structure, each user has their own *user files directory (UFD)*. The UFDs have similar structures, but each lists only the files of a single user. The system's *master file directory (MFD)* is searched whenever a new user id=s logged in. The MFD is indexed by username or account number, and each entry points to the UFD for that user.



Advantages:

- We can give full path like /User-name/directory-name/.
- Different users can have the same directory as well as the file name.
- Searching of files becomes easier due to pathname and user-grouping.

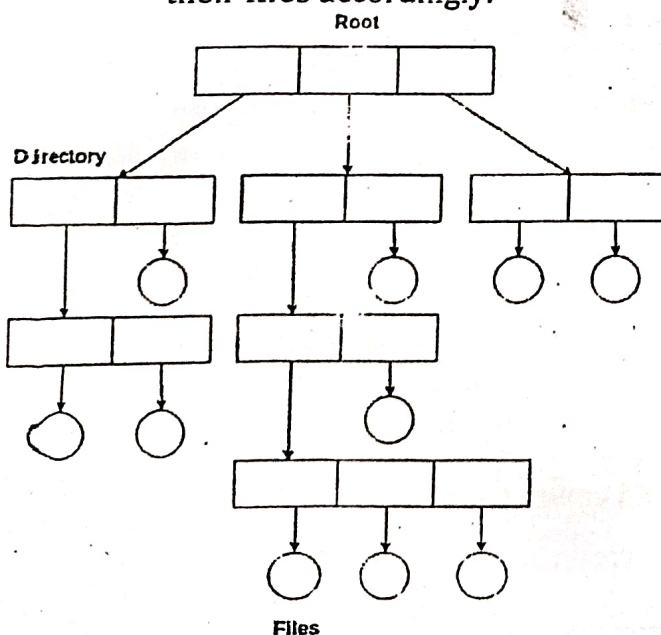
Disadvantages:

- A user is not allowed to share files with other users.
- Still, it not very scalable, two files of the same type cannot be grouped together in the same user.

• Tree-structured directory -

Once we have seen a two-level directory as a tree of height 2, the natural generalization is to extend the directory structure to a tree of arbitrary height.

This generalization allows the user to create their own subdirectories and to organize their files accordingly.



A tree structure is the most common directory structure. The tree has a root directory, and every file in the system has a unique path.

Advantages:

- Very general, since full pathname can be given.
- Very scalable, the probability of name collision is less.
- Searching becomes very easy, we can use both absolute paths as well as relative.

Disadvantages:

- Every file does not fit into the hierarchical model, files may be saved into multiple directories.
- We can not share files.
- It is inefficient, because accessing a file may go under multiple directories.

File Allocation Methods

The allocation methods define how the files are stored in the disk blocks. There are three main disk space or file allocation methods.

- Contiguous Allocation
- Linked Allocation
- Indexed Allocation

The main idea behind these methods is to provide:

- Efficient disk space utilization.
- Fast access to the file blocks.

All the three methods have their own advantages and disadvantages as discussed below:

1. Contiguous Allocation

In this scheme, each file occupies a contiguous set of blocks on the disk. For example, if a file requires n blocks and is given a block b as the starting location, then the blocks assigned to the file will be: $b, b+1, b+2, \dots, b+n-1$. This means that given the starting block address and the length of the file (in terms of blocks required), we can determine the blocks occupied by the file.

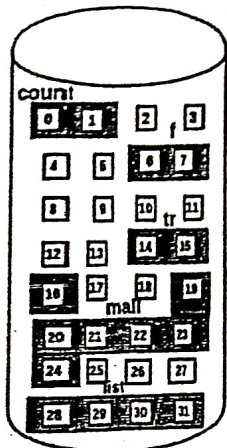
The directory entry for a file with contiguous allocation contains

- Address of starting block
- Length of the allocated portion.

The file 'mail' in the following figure starts from the block 19 with length = 6 blocks. Therefore, it occupies 19, 20, 21, 22, 23, 24 blocks.

Directory

file	start	length
count	0	2
p	14	3
mail	19	6
list	28	4
f	6	2



Advantages:

- Both the Sequential and Direct Accesses are supported by this. For direct access, the address of the k th block of the file which starts at block b can easily be obtained as $(b+k)$.

Disadvantages:

- This method suffers from both internal and external fragmentation. This makes it inefficient in terms of memory utilization.
- Increasing file size is difficult because it depends on the availability of contiguous memory at a particular instance.

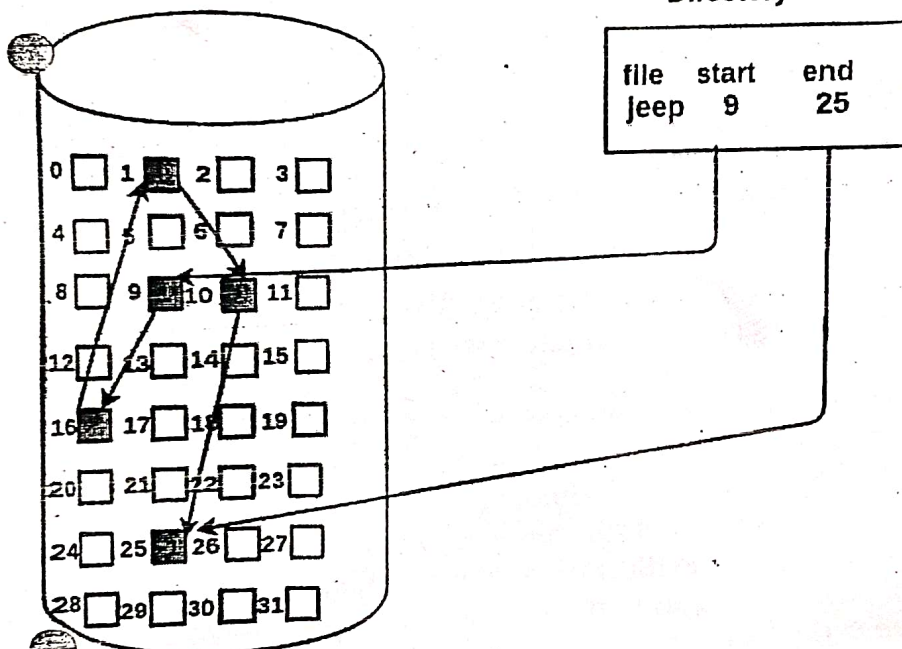
2. Linked List Allocation

In this scheme, each file is a linked list of disk blocks which **need not be** contiguous. The disk blocks can be scattered anywhere on the disk.

The directory entry contains a pointer to the starting and the ending file block. Each block contains a pointer to the next block occupied by the file.

The file 'jeep' in following image shows how the blocks are randomly distributed. The last block (25) contains -1 indicating a null pointer and does not point to any other block.

Directory



Advantages:

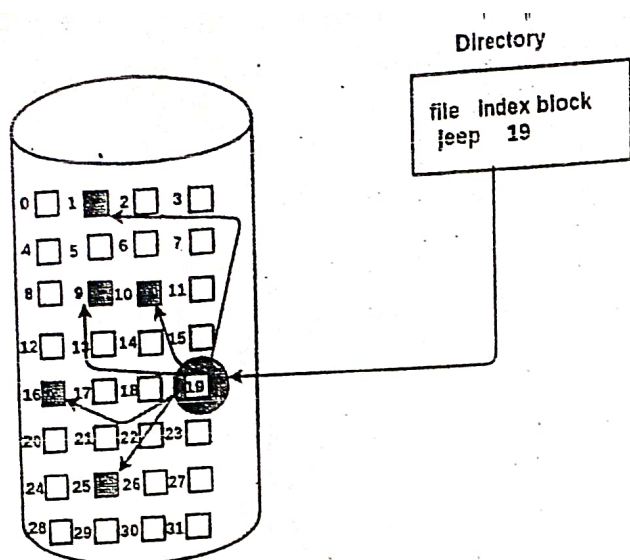
- This is very flexible in terms of file size. File size can be increased easily since the system does not have to look for a contiguous chunk of memory.
- This method does not suffer from external fragmentation. This makes it relatively better in terms of memory utilization.

Disadvantages:

- Because the file blocks are distributed randomly on the disk, a large number of seeks are needed to access every block individually. This makes linked allocation slower.
- It does not support random or direct access. We can not directly access the blocks of a file. A block k of a file can be accessed by traversing k blocks sequentially (sequential access) from the starting block of the file via block pointers.
- Pointers required in the linked allocation incur some extra overhead.

3. Indexed Allocation

In this scheme, a special block known as the **Index block** contains the pointers to all the blocks occupied by a file. Each file has its own index block. The *i*th entry in the index block contains the disk address of the *i*th file block. The directory entry contains the address of the index block as shown in the image:



Advantages:

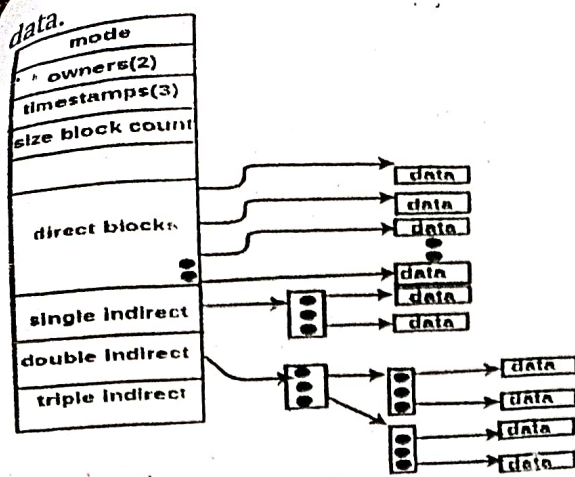
- This supports direct access to the blocks occupied by the file and therefore provides fast access to the file blocks.
- It overcomes the problem of external fragmentation.

Disadvantages:

- The pointer overhead for indexed allocation is greater than linked allocation.
- For very small files, say files that expand only 2-3 blocks, the indexed allocation would keep one entire block (index block) for the pointers which is inefficient in terms of memory utilization. However, in linked allocation we lose the space of only 1 pointer per block.

For files that are very large, single index block may not be able to hold all the pointers. Following mechanisms can be used to resolve this:

1. **Linked scheme:** This scheme links two or more index blocks together for holding the pointers. Every index block would then contain a pointer or the address to the next index block.
2. **Multilevel index:** In this policy, a first level index block is used to point to the second level index blocks which in turn points to the disk blocks occupied by the file. This can be extended to 3 or more levels depending on the maximum file size.
3. **Combined Scheme:** In this scheme, a special block called the **Inode (information Node)** contains all the information about the file such as the name, size, authority, etc and the remaining space of Inode is used to store the Disk Block addresses which contain the actual file as shown in the image below. The first few of these pointers in Inode point to the **direct blocks** i.e the pointers contain the addresses of the disk blocks that contain data of the file. The next few pointers point to indirect blocks. Indirect blocks may be single indirect, double indirect or triple indirect. **Single Indirect block** is the disk block that does not contain the file data but the disk address of the blocks that contain the file data. Similarly, **double indirect blocks** do not contain the file data but the disk address of the blocks that contain the address of the blocks containing the file



Free space management in Operating System

The system keeps tracks of the free disk blocks for allocating space to files when they are created. Also, to reuse the space released from deleting the files, free space management becomes crucial. The system maintains a free space list which keeps track of the disk blocks that are not allocated to some file or directory. The free space list can be implemented mainly as:

1. Bitmap or Bit vector -

A Bitmap or Bit Vector is series or collection of bits where each bit corresponds to a disk block. The bit can take two values: 0 and 1: 0 indicates that the block is allocated and 1 indicates a free block.

The given instance of disk blocks on the disk in Figure 1 (where green blocks are allocated) can be represented by a bitmap of 16 bits as: 0000111000000110.

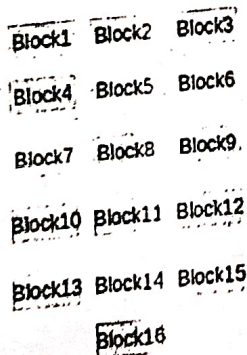


Figure - 1

Advantages -

- Simple to understand.
- Finding the first free block is efficient. It requires scanning the words (a group of 8 bits) in a bitmap for a non-zero word. (A 0-valued word has all bits 0). The first free block is then found by scanning for the first 1 bit in the non-zero word.

The block number can be calculated as:

$$(\text{number of bits per word}) * (\text{number of 0-values words}) + \text{offset of bit first bit 1 in the non-zero word}.$$

For the *Figure-1*, we scan the bitmap sequentially for the first non-zero word. The first group of 8 bits (00001110) constitute a non-zero word since all bits are not 0. After the non-0 word is found, we look for the first 1 bit. This is the 5th bit of the non-zero word. So, offset = 5. Therefore, the first free block number = $8 \times 0 + 5 = 5$.

2. Linked List -

In this approach, the free disk blocks are linked together i.e. a free block contains a pointer to the next free block. The block number of the very first disk block is stored at a separate location on disk and is also cached in memory.

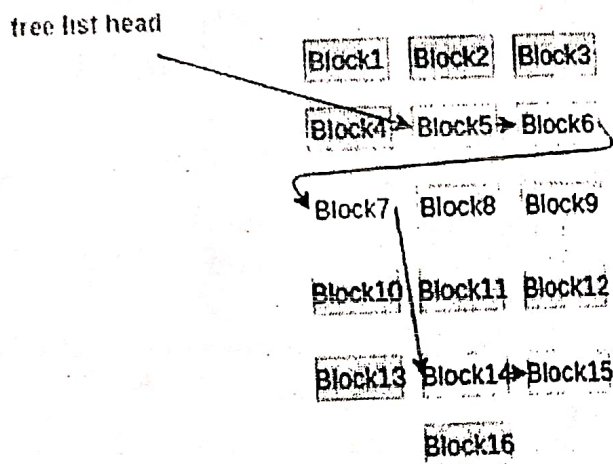


Figure - 2

In *Figure-2*, the free space list head points to Block 5 which points to Block 6, the next free block and so on. The last free block would contain a null pointer indicating the end of free list.

A drawback of this method is the I/O required for free space list traversal.

3. Grouping -

This approach stores the address of the free blocks in the first free block. The first free block stores the address of some, say n free blocks. Out of these n blocks, the first $n-1$ blocks are actually free and the last block contains the address of next free n blocks. An advantage of this approach is that the addresses of a group of free disk blocks can be found easily.

4. Counting -

This approach stores the address of the first free disk block and a number n of free contiguous disk blocks that follow the first block.

Every entry in the list would contain:

1. Address of first free disk block
2. A number n

For example, in *Figure-1*, the first entry of the free space list would be: ([Address of Block 5], 2), because 2 contiguous free blocks follow block 5.